ENTERPRISE PRESENTATION GUIDE

Shop Management System
API Architecture & Masterclass

এপিআই কী, কিভাবে সফটওয়্যারে কাজ করে, মেথডসমূহ (GET, POST, PUT, PATCH, DELETE), CRUD অপারেশন এবং কোড স্ট্রাকচারের পূর্ণাঙ্গ স্টেপ-বাই-স্টেপ গাইড

এই প্রেজেন্টেশনে যা যা কভার করা হয়েছে:

- ✓ **মডিউল ১:**API এর বেসিক পরিচিতি ও রিয়েল-লাইফ রেস্তোরাঁ অ্যানালজি
- ✓ **মডিউল ২:**HTTP Protocols, Verbs, Headers & Status Codes
- ✓ **মডিউল ৩:**FastAPI Swagger UI (`http://127.0.0.1:8000/docs`) এর কাজ ও ব্যবহার
- ✓ **মডিউল ৪:**আমাদের প্রজেক্টের কোড আর্কিটেকচার (কোন ফোল্ডার ও ফাইলে কি আছে)
- ✓ **মডিউল ৫:**সম্পূর্ণ ১৭টি এপিআই এন্ডপয়েন্টের পুঙ্খানুপুঙ্খ বিবরণ
- ✓ **মডিউল ৬:**বাস্তব সিনারিও (লগইন, স্টক অটো-ডিডাকশন, রিফান্ড ও রেভিনিউ লজিক)
- ✓ **মডিউল ৭:**ভাইভা / প্রেজেন্টেশন প্রশ্নোত্তর প্রস্তুতি (১০টি কোর প্রশ্ন)

Project: Shop Management System (FastAPI + PostgreSQL + Tailwind)

Target Audience: Presentation / Viva / Interview / Technical Review

মডিউল ১: API কী এবং সফটওয়্যারে এটি কিভাবে কাজ করে?

১.১ API এর সংজ্ঞা (What is an API?)

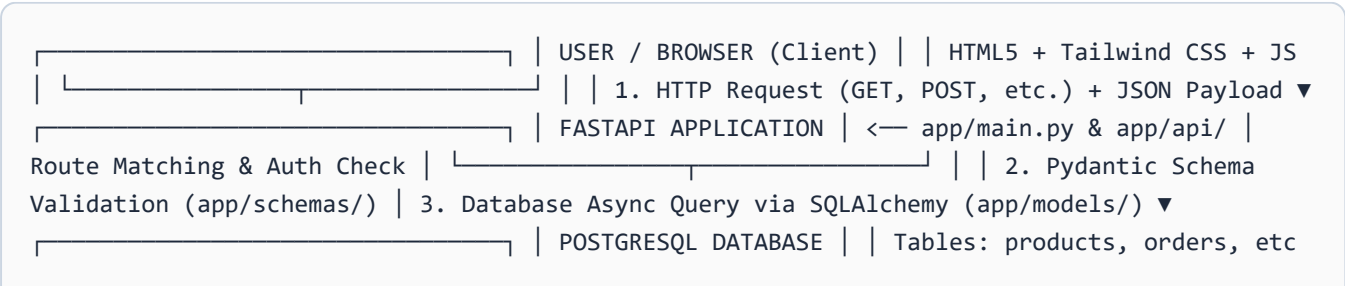
API এর পূর্ণরূপ হলো **Application Programming Interface**। সহজ বাংলায় API হলো একটি **ডিজিটাল সেতু বা বার্তাবাহক (Messenger)**, যার মাধ্যমে দুটি ভিন্ন সফটওয়্যার সিস্টেম (যেমন ব্রাউজারের ফ্রন্টএন্ড এবং সার্ভারের ব্যাকএন্ড) নিরাপদে একে অপরের সাথে ডেটা আদান-প্রদান করে।

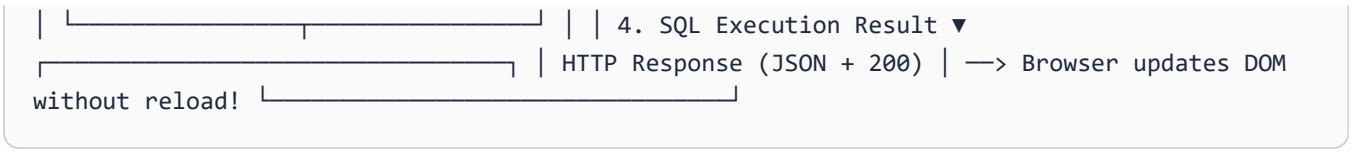
> 🍽️ **রিয়েল-লাইফ রেস্তোরাঁ অ্যানালজি (The Restaurant Waiter Analogy)**
> API বোঝার জন্য একটি রেস্তোরাঁর উদাহরণ সবচেয়ে সেরা:
>
> - **১. কাস্টমার (Customer) = ফ্রন্টএন্ড / ব্রাউজার (Frontend / Client):** আপনি চেয়ারে বসে মেনু কার্ড দেখে খাবার পছন্দ করেন এবং অর্ডার দেন।
> - **২. ওয়েটার (Waiter) = API (Application Programming Interface):** ওয়েটার আপনার অর্ডারটি খাতায় লিখে রান্নাঘরে পৌঁছে দেয় এবং রান্না শেষ হলে খাবার এনে আপনার টেবিলে পরিবেশন করে।
> - **৩. রান্নাঘর ও শেফ (Kitchen & Chef) = ব্যাকএন্ড ও ডেটাবেস (FastAPI & PostgreSQL):** সেখানে খাবার (ডেটা) প্রস্তুত থাকে এবং শেফ নিয়ম মেনে রান্না করে ওয়েটারকে দেয়।
>
> *এখানে কাস্টমার কখনো সরাসরি রান্নাঘরে ঢোকে না (সিকিউরিটি); ওয়েটার মাঝখানে নিয়মতান্ত্রিকভাবে কাজ করে—ঠিক একইভাবে ফ্রন্টএন্ড সরাসরি ডেটাবেস টাচ করে না, API এর মাধ্যমে যোগাযোগ করে।*

১.২ সফটওয়্যারে API কেন অপরিহার্য?

সুবিধা	কেন এটি জরুরি এবং কীভাবে কাজ করে?
১. ডিসেন্ট্রালাইজড আর্কিটেকচার	ফ্রন্টএন্ড (HTML/CSS/JS) এবং ব্যাকএন্ড (Python/FastAPI) সম্পূর্ণ আলাদা থাকে। ফলে ভবিষ্যতে মোবাইল অ্যাপ বা ডেস্কটপ অ্যাপ বানালেও একই ব্যাকএন্ড API ব্যবহার করা যাবে।
২. নিরাপত্তা (Security)	ডেটাবেসের পাসওয়ার্ড, সিক্রেট কি এবং আসল বিজনেস লজিক ক্লায়েন্টের কাছে প্রকাশ পায় না। API শুধুমাত্র অনুমোদিত রিকোয়েস্টে বৈধ ডেটা দেয়।
৩. রিঅ্যাক্টিভ ও ফাস্ট ইউজার এক্সপেরিয়েন্স	পুরো পেজ রিলোড না দিয়ে শুধুমাত্র ব্যাকগ্রাউন্ডে JSON ডেটা নিয়ে এসে স্ক্রিনের অংশবিশেষ পরিবর্তন করা যায় (SPA - Single Page Application স্টাইল)।

১.৩ ক্লায়েন্ট-সার্ভার আর্কিটেকচার ডায়াগ্রাম




| └──────────────┬──────────────┘ | | 4. SQL Execution Result ▼
┌──────────────────────────────┐ | HTTP Response (JSON + 200) | ──> Browser updates DOM
without reload! └──────────────────────────────┘

মডিউল ২: HTTP Protocols, Verbs, এবং CRUD অপারেশন

২.১ CRUD অপারেশন কী? (Create, Read, Update, Delete)

যেকোনো ডেটাবেস-চালিত অ্যাপ্লিকেশনে ডেটা পরিচালনার ৪টি মৌলিক কাজ থাকে, যাকে সংক্ষেপে **CRUD** বলা হয়। RESTful API তে এই ৪টি কাজের জন্য নির্দিষ্ট **HTTP Verbs (Methods)** ব্যবহার করা হয়:

CRUD অ্যাকশন	HTTP Method	আমাদের প্রজেক্টের রিয়েল উদাহরণ	কাজ ও বৈশিষ্ট্য
Create (নতুন তৈরি)	`POST`	`POST /api/products` `POST /api/orders`	সার্ভারে নতুন ডেটা যোগ করতে ব্যবহৃত হয়। সাথে ডেটার পেলোড (Body) পাঠাতে হয়। সফল হলে স্ট্যাটাস কোড **201 Created** ফেরত দেয়।
Read (ডেটা দেখা)	`GET`	`GET /api/products` `GET /api/dashboard/stats`	সার্ভার থেকে শুধু ডেটা পড়ার জন্য ব্যবহৃত হয়। এটি সার্ভারের কোনো ডেটা পরিবর্তন করে না (Idempotent & Safe)।
Update (সম্পূর্ণ পরিবর্তন)	`PUT`	`PUT /api/products/{id}` `PUT /api/customers/{id}`	একটি বিদ্যমান রেকর্ডের সমস্ত ফিল্ড একসাথে পরিবর্তন/প্রতিস্থাপন করতে ব্যবহৃত হয়।
Partial Update (আংশিক পরিবর্তন)	`PATCH`	`PATCH /api/orders/{id}/status`	পুরো রেকর্ড না বদলে শুধুমাত্র নির্দিষ্ট এক বা দুটি ফিল্ড আপডেট করতে ব্যবহৃত হয় (যেমন শুধুমাত্র অর্ডারের Status Delivered বা Cancelled করা)।
Delete (মুছে ফেলা)	`DELETE`	`DELETE /api/products/{id}` `DELETE /api/customers/{id}`	সার্ভার থেকে কোনো রেকর্ড মুছে ফেলার জন্য। আমাদের প্রজেক্টে ডেটা ইন্টিগ্রিটির জন্য এটি সফট-ডিলিট (`is_deleted = True`) করে।

২.২ একটি HTTP Request এর উপাদানসমূহ (Anatomy of a Request)

১ Request URL / Endpoint

সার্ভারের নির্দিষ্ট ঠিকানার পাথ। যেমন: `http://127.0.0.1:8000/api/products`

২ HTTP Method

অ্যাকশন নির্দেশ করে: `GET`, `POST`, `PUT`, `PATCH`, বা `DELETE`

৩ Headers (হেডার)

রিকোয়েস্টের অতিরিক্ত মেটা-তথ্য। যেমন: `Content-Type: application/json` (ডেটা ফরম্যাট) এবং `Cookie: access_token=...` (অথেন্টিকেশন পরিচয়)।

৪ Query Parameters

ইউআরএলের শেষে ? দিয়ে পাঠানো ফিল্টার বা পেজিনেশন তথ্য। যেমন: `/api/products?page=1&page_size=20&search=mouse`

৫ Request Body (JSON Payload)

সার্ভারে যে ডেটা পাঠাচ্ছেন। যেমন নতুন পণ্য তৈরির সময়:

```
{
  "name": "Wireless Mechanical Keyboard",
  "sku": "ELEC-KEY-001",
  "category": "Electronics",
  "price": 4500.00,
  "stock_quantity": 25,
  "reorder_level": 5
}
```

২.৩ স্ট্যান্ডার্ড HTTP Status Codes (যেগুলো সবার জানা উচিত)

কোড	নাম	অর্থ ও আমাদের প্রজেক্টে কখন আসে?
200	OK	রিকোয়েস্ট সফল হয়েছে এবং ডেটা রিটার্ন হয়েছে (যেমন প্রোডাক্ট লিস্ট বা ড্যাশবোর্ড স্ট্যাটস)।
201	Created	নতুন রেকর্ড সফলভাবে তৈরি হয়েছে (যেমন নতুন অর্ডার বা প্রোডাক্ট সফলভাবে ডেটাবেজে সেভ হয়েছে)।
400	Bad Request	ক্লায়েন্টের তথ্যে কোনো যুক্তিগত ভুল (যেমন ইতিমধ্যে ব্যবহৃত SKU দিয়ে প্রোডাক্ট অ্যাড করার চেষ্টা)।
401	Unauthorized	ব্যবহারকারী লগইন করেননি বা কুকিতে থাকা JWT টোকেন মেয়াদোত্তীর্ণ বা অবৈধ।
404	Not Found	যে আইডি চাওয়া হয়েছে তা ডেটাবেজে নেই (যেমন এমন কোনো অর্ডার আইডি যা বিদ্যমান নেই)।
422	Unprocessable Entity	Pydantic ডেটা ভ্যালিডেশন ব্যর্থ (যেমন দামের ঘরে সংখ্যার বদলে লেখা বা লিমিট অতিক্রম)।
500	Internal Server Error	সার্ভারের অভ্যন্তরীণ ক্র্যাশ বা ডেটাবেস কানেকশন ফেইলিয়র।

মডিউল ৩: FastAPI Swagger UI (http://127.0.0.1:8000/docs) কী?

৩.১ Swagger UI কী এবং এটি কিভাবে নিজে নিজে তৈরি হয়?

আপনি যখন ব্রাউজারে `http://127.0.0.1:8000/docs` ওপেন করেন, তখন আপনি একটি ইন্টারেক্টিভ ও চমৎকার ওয়েব পেজ দেখতে পান যেখানে আপনার অ্যাপের সব এপিআই তালিকাভুক্ত থাকে।

অন্যান্য ফ্রেমওয়ার্কে (যেমন Django বা Flask) এপিআই ডকুমেন্টেশন তৈরি করতে আলাদাভাবে প্রচুর কোড বা থার্ড-পার্টি টুল লিখতে হয়। কিন্তু **FastAPI এর অন্যতম ম্যাজিক হলো এটি সম্পূর্ণ স্বয়ংক্রিয়ভাবে OpenAPI স্পেসিফিকেশন থেকে Swagger UI রেন্ডার করে।**

> 💡 **কীভাবে এটি স্বয়ংক্রিয়ভাবে তৈরি হয়?**
>
> 1. আমরা যখন Python ফাংশনের উপরে `@router.get(...)` বা `@router.post(...)` লিখি, FastAPI তা ট্র্যাক করে।
> 2. ফাংশনে ব্যবহৃত **Pydantic Schemas** (যেমন `ProductCreate`, `ProductResponse`) থেকে FastAPI স্বয়ংক্রিয়ভাবে বুঝে নেয় কোন ফিল্ড রিকোয়ার্ড, কোনটা অপশনাল, কোনটা ইন্টিজার বা স্ট্রিং।
> 3. FastAPI ব্যাকগ্রাউন্ডে একটি `/openapi.json` ফাইল তৈরি করে, এবং Swagger UI সেটি ব্রাউজারে সুন্দর গ্রাফিক্যাল ইন্টারফেসে রূপান্তর করে।

৩.২ /docs পেজের মাধ্যমে ব্রাউজার থেকেই API টেস্ট করার নিয়ম

১ এন্ডপয়েন্ট এক্সপ্যান্ড করুন

যেকোনো এপিআই (যেমন `POST /api/products`) এর উপর ক্লিক করুন।

২ "Try it out" বাটনে ক্লিক করুন

ডানপাশে থাকা **"Try it out"** বাটনে ক্লিক করলে ইনপুট ফিল্ডগুলো এডিটেবল হয়ে যাবে।

৩ Request Body বা Parameters পূরণ করুন

প্রয়োজনীয় মান যেমন পণ্যের নাম, দাম, SKU ইত্যাদি লিখুন।

৪ "Execute" বাটন চাপুন

নীল রঙের **"Execute"** বাটনে ক্লিক করলেই ব্রাউজার স্বয়ংক্রিয়ভাবে আসল ডেটাবেসে রিকোয়েস্ট পাঠাবে এবং সাথে সাথে নিচের রেসপন্স বক্সে Status Code (200/201), Response Body (JSON), এবং Response Headers প্রদর্শন করবে।

> 📌 **বিকল্প ভিউ: ReDoc (`http://127.0.0.1:8000/redoc`)**

FastAPI স্বয়ংক্রিয়ভাবে **ReDoc** ইন্টারফেসও প্রদান করে, যা ক্লায়েন্ট বা থার্ড-পার্টি ডেভেলপারদের পড়ার উপযোগী একটি ক্লিন বুক-স্টাইলের ডকুমেন্টেশন।

মডিউল ৪: আমাদের প্রজেক্ট আর্কিটেকচার ও ফাইল কানেকশন ম্যাপ

৪.১ ফোল্ডার স্ট্রাকচার ও দায়িত্ব বিভাজন (Layered Architecture)

আমাদের প্রজেক্টটি একটি প্রফেশনাল ও মডুলার **Separation of Concerns** প্যাটার্নে তৈরি। নিচে প্রতিটি ফোল্ডার ও ফাইলের দায়িত্ব এবং কানেকশন ব্যাখ্যা করা হলো:

ফোল্ডার / ফাইল	লেয়ারের নাম	মূল কাজ ও সংযোগ
`app/main.py`	Application Entrypoint	FastAPI অ্যাপ তৈরি করে, ডাটাবেস পিং করে, টেবিল অটো-ক্রিয়েট করে, CORS ও স্ট্যাটিক ফাইল মাউন্ট করে এবং মাস্টার রাউটার যুক্ত করে।
`app/api/`	Controller / Router Layer	HTTP রিকোয়েস্ট রিসিভ করে, লজিক রান করে এবং JSON রেসপন্স পাঠায়। (auth, products, customers, orders, inventory, dashboard).
`app/core/database.py`	Database Engine & Session	PostgreSQL এর সাথে Async Engine তৈরি করে এবং `get_db` ডিপেন্ডেন্সির মাধ্যমে প্রতিটি রিকোয়েস্টে সেশন প্রদান করে।
`app/core/security.py`	Security & JWT Helper	পাসওয়ার্ড হ্যাশিং (bcrypt) এবং JWT অ্যাক্সেস টোকেন এনকোড/ডিকোড করে।
`app/core/deps.py`	Dependency Injection	`get_current_user` ডিপেন্ডেন্সি; এটি ব্রাউজারের HttpOnly কুকি থেকে টোকেন চেক করে ব্যবহারকারীকে শনাক্ত করে।
`app/models/`	SQLAlchemy ORM Models	PostgreSQL এর টেবিল স্কিমা ক্লাস হিসেবে সংজ্ঞায়িত (User, Product, Customer, Order, OrderItem, StockMovement)।
`app/schemas/`	Pydantic Data Validation	ক্লায়েন্ট থেকে আসা ডেটার ভ্যালিডেশন এবং রেসপন্স সিরিয়ালাইজেশনের ডেটা মডেল।
`app/templates/`	Frontend HTML (Jinja2)	ইউজারের ড্যাশবোর্ড, টেবিল স্ট্রাকচার, সাইডবার ও মডালসমূহ।
`app/static/js/app.js`	Client-Side Vanilla JS	ব্রাউজার থেকে API তে `fetch()` রিকোয়েস্ট পাঠিয়ে ড্যাশবোর্ডের ডেটা ডায়নামিকালি রেন্ডার করে।

৪.২ একটি রিকোয়েস্টের সম্পূর্ণ জীবনচক্র (Request Lifecycle Trace)

ধরা যাক ইউজার একটি পণ্য যুক্ত করতে "Save Product" বাটনে চাপ দিল। পুরো সিস্টেমে তখন যা ঘটে:

১ ফ্রন্টএন্ড অ্যাকশন (app/static/js/app.js)

ইউজার ফর্ম পূরণ করে সাবমিট দিলে জাভাস্ক্রিপ্ট ফর্মের ডেটা সংগ্রহ করে: `fetch('/api/products', { method: 'POST', body: JSON.stringify({...}) })`

২ রাউট ম্যাচিং (app/main.py -> app/api/products.py)

FastAPI রিকোয়েস্টটি ধরে `app/api/products.py` ফাইলের `@router.post("")` ফাংশনে পাঠায়।

৩ সিকিউরিটি ও ডিপেন্ডেন্সি চেক (app/core/deps.py)

`current_user: User = Depends(get_current_user)` এক্সিকিউট হয়। ব্রাউজারের কুকি থেকে JWT টোকেন ডিকোড করে নিশ্চিত হয় যে রিকোয়েস্টকারী একজন ভ্যালিড ইউজার।

৪ ডেটা ভ্যালিডেশন (app/schemas/product.py)

`product_in: ProductCreate` কার্যকর হয়। নাম ২ অক্ষরের বড় কি না, দাম পজিটিভ কি না এবং SKU ইউনিক কি না তা Pydantic দিয়ে ভ্যালিডেট হয়।

৫ ডাটাবেসে সেভ (app/models/product.py & app/core/database.py)

SQLAlchemy ORM মডেল দিয়ে `db.add(new_product)` এবং `await db.commit()` এক্সিকিউট করে PostgreSQL টেবিলে ডাটা পার্মানেন্টলি রাইট হয়।

৬ JSON রেসপন্স ও DOM আপডেট (app.js)

সার্ভার স্ট্যাটাস কোড **201 Created** সহ নতুন তৈরি পণ্যের JSON পাঠায়। ব্রাউজারের জাভাস্ক্রিপ্ট সেই রেসপন্স পেয়ে মডাল বন্ধ করে দেয় এবং টেবিল রিলোড না করেই পণ্যের সারি যুক্ত করে দেয়।

মডিউল ৫: আমাদের প্রজেক্টের সমস্ত এপিআই এন্ডপয়েন্ট ডিকশনারি

৫.১ Authentication APIs (`app/api/auth.py`)

Method & Route	Purpose (কাজ)	Input (প্যারামিটার/বডি)	Output / Status
`POST` `/api/auth/register`	প্রথমবার প্রথম অ্যাডমিন অ্যাকাউন্ট তৈরি (এরপর রেজিস্ট্রেশন লক হয়ে যায়)	`UserRegister` (full_name, email, password, confirm_password)	`201 Created` (User object)
`POST` `/api/auth/login`	ইমেইল ও পাসওয়ার্ড ভেরিফাই করে HttpOnly JWT কুকি সেট করে	`UserLogin` (email, password)	`200 OK` + `Set-Cookie: access_token=...`
`POST` `/api/auth/logout`	ইউজারের কুকি মুছে দিয়ে লগআউট করায়	None	`200 OK` + Cookie clear
`GET` `/api/auth/me`	বর্তমান লগইন থাকা ইউজারের প্রোফাইল তথ্য দেয়	None (Requires Auth Cookie)	`200 OK` (User info)
`GET` `/api/auth/status`	অ্যাডমিন অ্যাকাউন্ট আছে কি না চেক করে (/register পেজ লকের জন্য)	None (Public)	`200 OK` (`{"has_admin": true/false}`)

৫.২ Product Management APIs (`app/api/products.py`)

Method & Route	Purpose (কাজ)	Input	Output / Status
`GET` `/api/products`	প্রোডাক্টের তালিকা, পেজিনেশন, সার্চ ও ক্যাটাগরি ফিল্টার	`?page=1&page_size=20&search=...&category=...`	`200 OK` (items list, total, categories)
`POST` `/api/products`	নতুন পণ্য যোগ করা (SKU	`ProductCreate` JSON Body	`201 Created`

Method & Route	Purpose (কাজ)	Input	Output / Status
	ইউনিকনেস ও পজিটিভ ভ্যালু চেক)		(New Product)
GET /api/products/{id}	নির্দিষ্ট পণ্যের বিস্তারিত তথ্য ফেচ করা	Path: `id` (int)	`200 OK` / `404 Not Found`
PUT /api/products/{id}	পণ্যের তথ্য আপডেট করা	Path: `id`, Body: `ProductUpdate`	`200 OK` (Updated Product)
DELETE /api/products/{id}	পণ্য সফট-ডিলিট করা (যদি কোনো অর্ডারে থাকে তবে ব্লক করে)	Path: `id`	`200 OK` (Success message)

৫.৩ Customer Management APIs (`app/api/customers.py`)

Method & Route	Purpose (কাজ)	Input	Output / Status
GET /api/customers	কাস্টমারদের তালিকা ও মোট অর্ডার সংখ্যা প্রদর্শন	`?page=1&page_size=20&search=...`	`200 OK` (Customer list with orders_count)
POST /api/customers	নতুন কাস্টমার নিবন্ধন	`CustomerCreate` (name, phone, email, address)	`201 Created`
PUT /api/customers/{id}	কাস্টমার প্রোফাইল আপডেট	Path: `id`, Body: `CustomerUpdate`	`200 OK`
DELETE /api/customers/{id}	কাস্টমার ডিলিট (সফট-ডিলিট)	Path: `id`	`200 OK`
GET /api/customers/{id}/orders	নির্দিষ্ট কাস্টমারের অতীতের সব অর্ডারের হিস্টোরি দেখা	Path: `id`	`200 OK` (Order history list)

মডিউল ৫ (চলমান): Orders, Inventory ও Dashboard APIs

৫.৪ Order Management APIs (`app/api/orders.py`)

Method & Route	Purpose (কাজ)	Business Logic & Impact
`GET` `/api/orders`	অর্ডার তালিকা দেখা, স্ট্যাটাস ও পেমেন্ট ফিল্টারিং	`?page=1&page_size=15&status=...&payment=...` সাপোর্ট করে।
`POST` `/api/orders`	নতুন মাল্টি-আইটেম সেলস অর্ডার তৈরি	**অটো-স্টক ডিডাকশন:** প্রতিটি আইটেমের জন্য স্টক যাচাই করে, অর্ডার তৈরি করে, পণ্যের স্টক থেকে কোয়ান্টিটি কমিয়ে দেয় এবং অডিট ট্রেইলে `OUT` মুভমেন্ট লগ তৈরি করে।
`PATCH` `/api/orders/{id}/status`	অর্ডারের স্ট্যাটাস বা পেমেন্ট পরিবর্তন	**অটো-স্টক রেস্টোরেশন:** কোনো অর্ডার `Cancelled` করা হলে পণ্যের স্টক ফিরিয়ে দেওয়া হয় এবং `IN` মুভমেন্ট লগ তৈরি হয়।
`GET` `/api/orders/{id}/invoice`	অফিসিয়াল কমার্শিয়াল ইনভয়েস ডেটা ফেচ	ইনভয়েস নম্বর, ক্রেতার তথ্য, আইটেমভিত্তিক ব্রেকডাউন ও মোট টাকার ফরম্যাটেড ডেটা প্রদান করে।

৫.৫ Inventory & Stock Movements (`app/api/inventory.py`)

Method & Route	Purpose (কাজ)	Business Logic & Impact
`GET` `/api/inventory/movements`	স্টক মুভমেন্টের অডিট ট্রেইল দেখা	কে কোন তারিখে কোন পণ্যের কত স্টক পরিবর্তন করেছে, তার পূর্ণ হিস্টোরি (টাইপ, পরিমাণ, আগের স্টক, বর্তমান স্টক, নোট)।
`POST` `/api/inventory/movement`	ম্যানুয়াল স্টক ইন, আউট বা অ্যাডজাস্টমেন্ট	**ট্রানজ্যাকশন সেফ:** `IN` হলে স্টক বাড়ে, `OUT` হলে স্টক কমে, `ADJUSTMENT` হলে সঠিক স্টক সেট হয়ে ডেটাবেসে পার্মানেন্টলি লগ থাকে।

৫.৬ Dashboard & Analytics APIs (`app/api/dashboard.py`)

Method & Route	Purpose (কাজ)	Business Logic & Impact
`GET` `/api/dashboard/stats`	ড্যাশবোর্ডের ওভারভিউ কার্ড ও ৭ দিনের চার্ট ডেটা	১. মোট সক্রিয় পণ্য সংখ্যা ২. মোট নিবন্ধিত কাস্টমার সংখ্যা ৩. মোট অর্ডার সংখ্যা **৪. Total Revenue:** শুধুমাত্র যেসব অর্ডারের `order_status == 'Delivered'` এবং `payment_status == 'Paid'`, তাদের টাকার যোগফল! ৫. Low Stock পণ্যের সংখ্যা (সাইডবার ব্যাজের জন্য) ৬. বিগত ৭ দিনের ডেলিভার্ড সেলস চার্ট ডেটা।
`GET` `/api/dashboard/low-stock`	কম স্টকের পণ্যের তালিকা	যেসব পণ্যের `stock_quantity <= reorder_level`, সেগুলোকে সবচেয়ে কম স্টক থেকে ক্রমানুসারে সাজিয়ে রিটার্ন করে।

মডিউল ৬: প্রেজেন্টেশনে বোঝানোর জন্য বাস্তব সিনারিও

সিনারিও ১: সিকিউর লগইন এবং HttpOnly JWT কুকি ফ্লো

১ ইউজার ইনপুট ও সাবমিট

ইউজার `/login` পেজে ইমেইল ও পাসওয়ার্ড লিখে সাবমিট দিলে `POST /api/auth/login` এ রিকোয়েস্ট যায়।

২ পাসওয়ার্ড ভেরিফিকেশন (Bcrypt)

`app/core/security.py` ডেটাবেসের হ্যাশড পাসওয়ার্ডের সাথে ইউজারের পাসওয়ার্ড মিলিয়ে দেখে।

৩ JWT টোকেন তৈরি ও HttpOnly কুকি সেট

পাসওয়ার্ড সঠিক হলে `create_access_token()` ফাংশন দিয়ে সাইন করা JWT তৈরি হয় এবং `response.set_cookie(key="access_token", httponly=True, samesite="lax")` রেসপন্সে পাঠানো হয়।

> 🔒 **কেন HttpOnly Cookie?** সাধারণ `localStorage` এ টোকেন রাখলে ব্রাউজারের কোনো ক্ষতিকর জাভাস্ক্রিপ্ট (XSS অ্যাটাক) টোকেন চুরি করতে পারে। কিন্তু `HttpOnly` কুকিকে জাভাস্ক্রিপ্ট কখনোই পড়তে পারে না, ফলে অ্যাপ অত্যন্ত সুরক্ষিত থাকে!

সিনারিও ২: সেলস অর্ডার তৈরি ও স্বয়ংক্রিয় স্টক হ্রাস (ACID Transaction)

যখন কোনো কাস্টমার অর্ডার দেয় (যেমন ৩টি কীবোর্ড), তখন ডেটাবেস ইনকনসিস্টেন্সি যেন কখনো না হয় সেজন্য এটি একটি **Atomic Transaction** এ ঘটে:

১ স্টক যাচাই (Stock Availability Check)

যদি প্রোডাক্টের বর্তমান স্টক ৩টির কম থাকে, তাহলে সাথে সাথে `400 Bad Request` পাঠায়: *"Insufficient stock for Keychron K2. Available: 2, Requested: 3"*।

২ স্টক কর্তন (Stock Deduction)

স্টক পর্যাপ্ত থাকলে পণ্যটির স্টক ৩ ইউনিট কমিয়ে দেওয়া হয়: `product.stock_quantity -= 3`।

৩ অডিট ট্রেইল তৈরি (Stock Movement Audit)

`StockMovement` টেবিলে একটি `OUT` রেকর্ড তৈরি হয়: কে অর্ডার করেছে, আগের স্টক কত ছিল, বর্তমান স্টক কত।

৪ কমিট (Database Commit)

সবগুলো অপারেশন সফল হলে একসাথে `await db.commit()` হয়। কোনো একটি স্টেপে ভুল হলে স্বয়ংক্রিয়ভাবে রোলব্যাক হয়, ফলে ভুল ডেটা সেভ হতে পারে না।

সিনারিও ৩: অর্ডার বাতিল হলে স্টক ফেরত আসা (Order Cancellation Refund)

কোনো কারণে গ্রাহক অর্ডার বাতিল করলে ড্যাশবোর্ডে স্ট্যাটাস `Cancelled` সেট করা হয়। ব্যাকএন্ডে `app/api/orders.py` স্বয়ংক্রিয়ভাবে শনাক্ত করে যে অর্ডারটি পূর্বে ক্যান্সেলড ছিল না, তাই অর্ডারে থাকা সমস্ত পণ্যের স্টক ফেরত দিয়ে দেয় এবং অডিট লগে `IN` মুভমেন্ট রেকর্ড করে।

সিনারিও ৪: শর্তসাপেক্ষ রেভিনিউ গণনা (Delivered & Paid Logic)

আমাদের অ্যাপে যেকোনো পেন্ডিং বা ক্যান্সেলড অর্ডারের টাকা রেভিনিউ কার্ডে যোগ হয় না। `app/api/dashboard.py` তে এক্সক্লুসিভ কুয়েরি চালানো হয়:

```
sales_res = await db.execute(
    select(func.sum(Order.total_amount)).where(
        Order.order_status == "Delivered",
        Order.payment_status == "Paid",
    )
)
```

এর ফলে একজন দোকানদার শুধুমাত্র তার হাতে আসা নিশ্চিত নগদ লাভ ও আসল বিক্রয় দেখতে পান।

মডিউল ৭: ভাইভা / প্রেজেন্টেশন প্রশ্নোত্তর প্রস্তুতি (Top 10 Q&A)

প্রশ্ন ১: RESTful API কী এবং এর বৈশিষ্ট্য কী?

উত্তর: REST (Representational State Transfer) হলো ওয়েব এপিআই ডিজাইনের একটি স্ট্যান্ডার্ড আর্কিটেকচারাল স্টাইল। এর প্রধান বৈশিষ্ট্য হলো এটি Stateless (প্রতিটি রিকোয়েস্ট স্বয়ংসম্পূর্ণ), ক্লায়েন্ট ও সার্ভারকে আলাদা রাখে, স্ট্যান্ডার্ড HTTP Verbs (GET, POST, PUT, DELETE) ব্যবহার করে এবং সাধারণত JSON ফরম্যাটে ডেটা আদান-প্রদান করে।

প্রশ্ন ২: FastAPI কেন অন্যান্য ফ্রেমওয়ার্ক (যেমন Flask বা Django) থেকে আলাদা বা দ্রুত?

উত্তর: FastAPI তৈরি হয়েছে আধুনিক Python 3.10+ এর টাইপ হিন্টস এবং Starlette ও Pydantic এর উপর। এটি সম্পূর্ণ Asynchronous (async/await), যার ফলে নোডজেএস বা গো (Go) ল্যাঙ্গুয়েজের মতো উচ্চ পারফরম্যান্স দেয়। এছাড়াও এটি কোড দেখেই নিজে নিজে Swagger UI ডকুমেন্টেশন তৈরি করে ফেলে।

প্রশ্ন ৩: Pydantic স্কিমা এবং SQLAlchemy মডেলের মধ্যে পার্থক্য কী?

উত্তর: SQLAlchemy Model ডেটাবেস টেবিলের গঠন নির্ধারণ করে (যেমন কোন কলাম Foreign Key, কলাম টাইপ ইত্যাদি)। আর Pydantic Schema নেটওয়ার্কের মাধ্যমে ক্লায়েন্ট ও সার্ভারের মধ্যে আদান-প্রদান হওয়া ডেটা যাচাই (Validate) ও রূপান্তর করে। যেমন ইউজারের পাসওয়ার্ড ডেটাবেসে থাকে কিন্তু রেসপন্স স্কিমায় পাসওয়ার্ড ফিল্ড বাদ দিয়ে পাঠানো হয়।

প্রশ্ন ৪: আপনার প্রজেক্টে ডেটাবেস কানেকশন কিভাবে হ্যান্ডেল করা হয়েছে?

উত্তর: আমরা SQLAlchemy 2.0 এর `AsyncEngine` এবং `asyncpg` ড্রাইভার ব্যবহার করেছি। `get_db()` নামক একটি অ্যাসিনক্রোনাস জেনারেটরের মাধ্যমে Dependency Injection ব্যবহার করে প্রতিটি রিকোয়েস্টে একটি নিরাপদ সেশন ওপেন হয় এবং রিকোয়েস্ট শেষ হলে স্বয়ংক্রিয়ভাবে সেশন ক্লোজ হয়।

প্রশ্ন ৫: PUT এবং PATCH এর মধ্যে পার্থক্য কী? আপনার প্রজেক্টে কোথায় কোনটি ব্যবহৃত হয়েছে?

উত্তর: `PUT` পুরো রেকর্ডকে সম্পূর্ণ নতুন ডেটা দিয়ে প্রতিস্থাপন করে (যেমন `PUT /api/products/{id}` পুরো পণ্যের নাম, দাম, স্টক একসাথে আপডেট করে)। আর `PATCH` শুধুমাত্র নির্দিষ্ট কিছু ফিল্ড আংশিক আপডেট করে (যেমন `PATCH /api/orders/{id}/status` শুধুমাত্র অর্ডারের স্ট্যাটাস পরিবর্তন করে)।

প্রশ্ন ৬: ডিপেন্ডেন্সি ইনজেকশন (Dependency Injection) কী?

উত্তর: ডিপেন্ডেন্সি ইনজেকশন হলো এমন একটি টেকনিক যেখানে একটি ফাংশন তার প্রয়োজনীয় ডিপেন্ডেন্সি নিজে তৈরি না করে বাইরে থেকে প্যারামিটার হিসেবে গ্রহণ করে। যেমন FastAPI তে `db: AsyncSession = Depends(get_db)` এবং `current_user: User = Depends(get_current_user)` লিখে আমরা ডাটাবেস সেশন ও অথেন্টিকেশন ম্যানেজ করেছি।

প্রশ্ন ৭: আপনার অ্যাপে Low Stock মনিটরিং কিভাবে কাজ করে?

উত্তর: প্রতিটি পণ্যের একটি `reorder_level` নির্ধারিত থাকে। ডেটাবেস কুয়েরির মাধ্যমে যেসব পণ্যের `stock_quantity <= reorder_level`, সেগুলোকে আলাদা করে সাইডবারে লাইভ ব্যাজ কাউন্ট দেখানো হয় এবং কম স্টকের তালিকায় সতর্কবার্তা প্রদর্শন করা হয়।

প্রশ্ন ৮: JWT টোকেন কিভাবে কাজ করে?

উত্তর: JWT (JSON Web Token) ৩টি অংশ নিয়ে গঠিত—Header, Payload এবং Signature। ইউজার লগইন করলে সার্ভার ইউজারের আইডি ও এক্সপায়ারি টাইম দিয়ে একটি গোপন কি (Secret Key) দ্বারা সাইন করা টোকেন প্রদান করে। পরবর্তী প্রতিটি রিকোয়েস্টে এই সিগনেচার যাচাই করে সার্ভার বুঝতে পারে ইউজার আসল কি না।

প্রশ্ন ৯: ইনভয়েস প্রিন্ট করার সময় ডেটা কিভাবে তৈরি হয়?

উত্তর: যখন ইউজার "Invoice" বাটনে ক্লিক করে, `GET /api/orders/{id}/invoice` এপিআই থেকে সংশ্লিষ্ট অর্ডারের সম্পূর্ণ বিবরণী JSON আকারে আসে। প্রিন্ট করার সময় একটি আইসোলেটেড প্রিন্ট ইঞ্জিন চালু হয়ে পরিচ্ছন্ন A4 পেপারে হাই-কন্ট্রাস্ট কমার্শিয়াল ইনভয়েস তৈরি করে।

প্রশ্ন ১০: প্রজেক্টে কনকারেন্ট অর্ডারিং বা ডেটা অসামঞ্জস্য রোধ করার উপায় কী?

উত্তর: আমরা ডেটাবেস ট্রানজ্যাকশন ব্যবহার করেছি। একাধিক ইউজার একসাথে অর্ডার করলেও স্টক ডিডাকশন এবং অর্ডার আইটেম তৈরি—এ ট্রানজ্যাকশনে থাকে, ফলে কখনোই স্টকের বেশি পণ্য বিক্রি হওয়া বা আংশিক ডেটা সেভ হওয়া সম্ভব নয়।
